

SCA TME 5 Attaque par Analyse de Corrélation de Consommation (CPA)

Delmotte Aurélien
Zhang Xiru

September 25, 2025

Contents

1	Introduction	2
2	Implémentation de l'attaque CPA	2
2.1	CPA en utilisant POIs	2
2.2	CPA 'online'	3
3	Comparaison entre DPA et CPA	4
3.1	L'implémentation de DPA Multi-bit en C	4
3.2	Comparaison des taux de réussite	4
3.3	Comparaison des temps d'exécution	5
4	Conclusion	7

1 Introduction

Les attaques par canaux auxiliaires, telles que l'Analyse de Puissance par Corrélation (CPA) et l'Analyse de Puissance Différentielle (DPA), jouent un rôle clé dans l'évaluation de la sécurité des systèmes cryptographiques modernes. Ces méthodes exploitent des informations indirectes, comme la consommation d'énergie, pour extraire des clés cryptographiques. Cependant, leur efficacité dépend fortement de deux facteurs principaux : le taux de réussite de l'extraction des clés et le temps nécessaire pour exécuter l'attaque.

Dans ce rapport, nous réalisons et comparons les performances des algorithmes CPA, CPA 'online' et DPA multibits en termes de taux de réussite et de temps d'exécution. Les implémentations en C et en Python ont été étudiées pour évaluer l'impact du choix du langage de programmation sur ces deux critères. Le CPA, souvent considéré comme une référence en cryptanalyse, est comparé à sa version CPA 'online' et au DPA multibits, une approche complémentaire mais plus complexe. L'objectif de ce travail est de fournir des recommandations pratiques pour optimiser les performances des attaques tout en garantissant leur efficacité dans différents contextes.

2 Implémentation de l'attaque CPA

2.1 CPA en utilisant POIs

L'analyse de puissance par corrélation (CPA) est une technique d'attaque par canaux auxiliaires qui exploite les variations de la consommation d'énergie pour obtenir des informations sur les clés cryptographiques. Dans cette section, nous détaillons notre implémentation de l'attaque CPA en utilisant les points d'intérêt (POIs) identifiés à partir des traces de puissance.

Le CPA se base sur le calcul de la corrélation entre les valeurs mesurées de consommation d'énergie et les valeurs prédites basées sur des hypothèses de clé. Pour chaque trace de puissance, nous calculons le coefficient de Pearson pour mesurer la corrélation entre la consommation d'énergie observée et celle prédite par le modèle cryptographique, comme décrit par l'équation suivante:

$$r_k = \frac{\sum_{n=1}^N (h_{n,k} - \bar{h}_k)(t_n - \bar{t})}{\sqrt{\sum_{n=1}^N (h_{n,k} - \bar{h}_k)^2} \sqrt{\sum_{n=1}^N (t_n - \bar{t})^2}}$$

où t_n représente la valeur de consommation de la trace n au point d'intérêt, $h_{n,k}$ est la valeur estimée de la consommation pour la trace n et l'hypothèse de clé k , $\bar{h}_k$ et $\bar{t}$ sont les moyennes des valeurs modélisées et mesurées, respectivement.

Pour mettre en pratique l'attaque, nous avons sélectionné des sous-ensembles de traces et exécuté l'analyse de corrélation sur des fenêtres de traces restreintes pour identifier les moments où la clé est le plus susceptible d'être exposée.

```
1 uint8_t guess_byte_CPA(int byte, int start, int end){
2     int poi = poi[byte];
3     double tbar = 0;
4     double tnt2 = 0;
5     double rkmax = 0;
6     uint8_t kmax = 0;
7     for(int n=start; n<end; n++){
8         tbar += TR(n)[poi];
9     }
10    tbar /= end - start;
11    for(int n=start; n<end; n++){
12        double ttnt = TR(n)[poi] - tbar;
13        tnt2 += ttnt * ttnt;
14    }
15    for(uint16_t k=0; k<0x100; k++){
16        double hbar = 0;
17        double hnh = 0;
18        double hnh2 = 0;
19        for(int n=start; n<end; n++){
20            hbar += PW(SBX(TX(n)[byte], k));
21        }
```

```

22     hbar/=end-start;
23     for(int n=start; n<end; n++){
24         double thnh=PW(SBX(TX(n)[byte], k))-hbar;
25         hnh += thnh*(TR(n)[poi]-tbar);
26         hnh2 += thnh*thnh;
27     }
28     double rk = 0;
29     if (sqrt(hnh2 * tnt2) != 0) {
30         rk = hnh / sqrt(hnh2 * tnt2);
31     }
32     if (fabs(rk)>rkmax){
33         rkmax=fabs(rk);
34         kmax=k;
35     }
36 }
37 return kmax;
38 }
39 }

```

Listing 1: CPA Implementation en C

2.2 CPA 'online'

L'approche *CPA 'online'* permet de traiter les traces de consommation en un seul passage permettant potentiellement de traiter les données en temps réel, sans nécessité de sauvegarde des traces. Cette méthode utilise une formule spécifique pour le calcul de corrélation qui prend en compte les valeurs accumulées au fur et à mesure des lectures des traces (Cette formule étant équivalente à celle de la CPA classique).

La corrélation en ligne entre la consommation d'énergie mesurée et les prédictions basées sur les hypothèses de clé est calculée par la formule suivante:

$$r_{k,i} = \frac{N \sum_{n=1}^N h_{n,k} t_{n,i} - \sum_{n=1}^N h_{n,k} \sum_{n=1}^N t_{n,i}}{\sqrt{(N \sum_{n=1}^N h_{n,k}^2 - (\sum_{n=1}^N h_{n,k})^2)(N \sum_{n=1}^N t_{n,i}^2 - (\sum_{n=1}^N t_{n,i})^2)}}$$

où N est le nombre de traces traitées jusqu'à présent, $h_{n,k}$ est la valeur estimée de la consommation pour la trace n et l'hypothèse de clé k , et $t_{n,i}$ est la valeur mesurée de la consommation au point d'intérêt i .

En utilisant des points de contrôle (*checkpoints*) et des seuils (*seuils*), l'algorithme évalue périodiquement la force des corrélations entre les hypothèses de clé et les mesures de consommation. Cette évaluation permet de terminer prématurément l'analyse si une corrélation suffisamment forte est détectée, économisant ainsi du temps et des ressources.

```

1  uint8_t guess_online_CPA(int byte, int start, int end) {
2      int poi = poil[byte];
3      double corr = 0;
4      uint8_t kmax = 0;
5      double sum_hnktni, sum_hnk, sum_tni, sum_h2nk, sum_t2ni;
6      for (uint16_t k = 0; k < 0x100; k++) {
7          sum_hnktni = sum_hnk = sum_tni = sum_h2nk = sum_t2ni = 0;
8          for (int n = start; n<end; n++) {
9              double hnk = PW(SBX(TX(n)[byte], k));
10             double tni = TR(n)[poi];
11             sum_hnktni += hnk * tni;
12             sum_hnk += hnk;
13             sum_tni += tni;
14             sum_h2nk += hnk * hnk;
15             sum_t2ni += tni * tni;
16             if ((n - start + 1) % CHECKPOINT == 0 || n == end - 1) {
17                 double rki = (sum_hnktni - (sum_hnk * sum_tni / (n-start + 1))) /
sqrt((sum_h2nk - pow(sum_hnk, 2) / (n-start + 1)) * (sum_t2ni - pow(sum_tni, 2)
/ (n-start + 1)));
18                 if (fabs(rki) > corr) {
19                     corr = fabs(rki);
20                     kmax = k;

```

```

21         if (corr > 0.9) {
22             return kmax;
23         }
24     }
25 }
26 }
27 }
28 return kmax;
29 }

```

Listing 2: CPA online Implementation en C

3 Comparaison entre DPA et CPA

3.1 L'implémentation de DPA Multi-bit en C

La version multibits de l'analyse différentielle de puissance (DPA) que nous avons implémentée est conçue pour exploiter les variations de consommation d'énergie en fonction du poids de Hamming du résultat intermédiaire, après l'application de la S-box avec chaque clé candidate. Le pseudo-code suivant décrit notre méthode:

```

1 // Pseudo-code pour DPA multibits
2 uint8_t guess_multi_DPA(int byte, int start, int end) {
3     int poi = poil[byte]; // POIs
4     double avg1 = 0, avg2 = 0; // Moyennes pour les groupes de poids de Hamming
5     int count1 = 0, count2 = 0; // Compteurs pour les groupes
6     double max_diff = 0; // Difference maximale observee
7     uint8_t kmax = 0; // Meilleure cle candidate
8
9     for (uint16_t key = 0; key < 0x100; key++) {
10         avg1 = avg2 = 0;
11         count1 = count2 = 0;
12         for (int n = start; n < end; n++) {
13             double hw = PW(SBX(TX(n)[byte], key)); // Poids de Hamming
14             if (hw < 4) {
15                 avg1 += TR(n)[poi]; // Accumulation pour le groupe 1
16                 count1++;
17             } else {
18                 avg2 += TR(n)[poi]; // Accumulation pour le groupe 2
19                 count2++;
20             }
21         }
22         avg1 /= count1; avg2 /= count2; // Calcul des moyennes
23         double diff = fabs(avg1 - avg2); // Calcul de la difference
24         if (diff > max_diff) {
25             max_diff = diff;
26             kmax = key;
27         }
28     }
29     return kmax;
30 }

```

Listing 3: Implémentation de DPA multibits en C

Cette approche par segmentation selon le poids de Hamming permet une distinction plus nette entre les différentes hypothèses de clés, augmentant potentiellement le taux de réussite de l'attaque.

3.2 Comparaison des taux de réussite

La Figure 1 illustre le taux de réussite des trois méthodes d'attaque CPA, CPA 'online', et DPA multibits en fonction du nombre de traces utilisées. Comme montré, le taux de réussite augmente avec le nombre de traces pour toutes les méthodes, mais avec des différences notables dans l'efficacité et la rapidité de convergence vers un taux de réussite élevé.

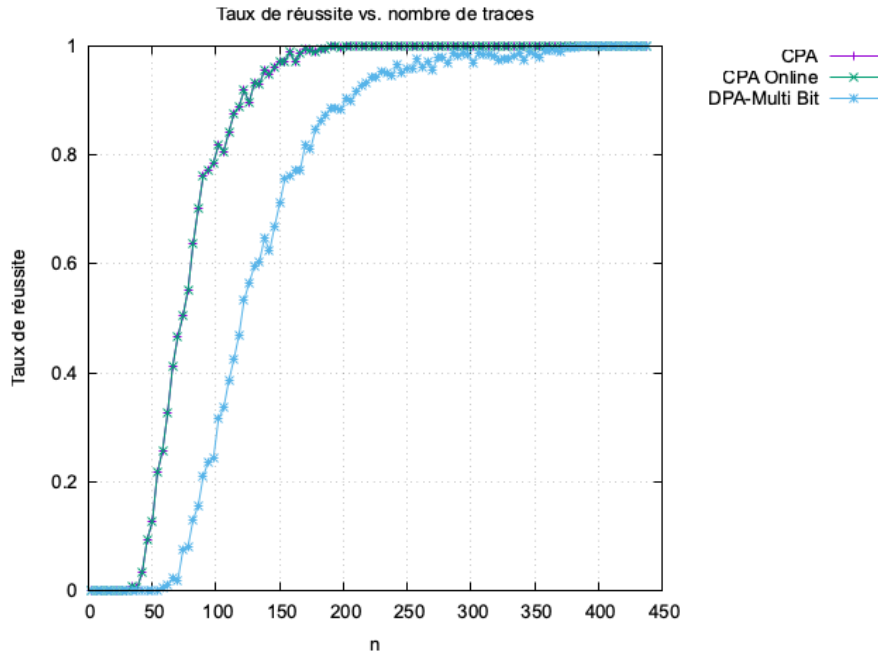


Figure 1: Comparaison des taux de réussite entre DPA multibits, CPA, et CPA 'online' en fonction du nombre de traces

Les courbes pour CPA et CPA 'online' coïncident bel et bien, indiquant bien que les deux algorithmes sont équivalents.

En revanche, il est notable que la méthode DPA multibits requiert un nombre plus élevé de traces pour atteindre des taux de réussite similaires à ceux des méthodes CPA. La méthode CPA est donc plus efficace nécessitant moins de données pour obtenir un résultat équivalent.

3.3 Comparaison des temps d'exécution

Nous avons évalué les performances en termes de temps d'exécution pour les algorithmes CPA, CPA 'online' et DPA multibits en C, et DPA multibits en Python. Les résultats de ces évaluations sont illustrés dans les Figures 2 et 3.

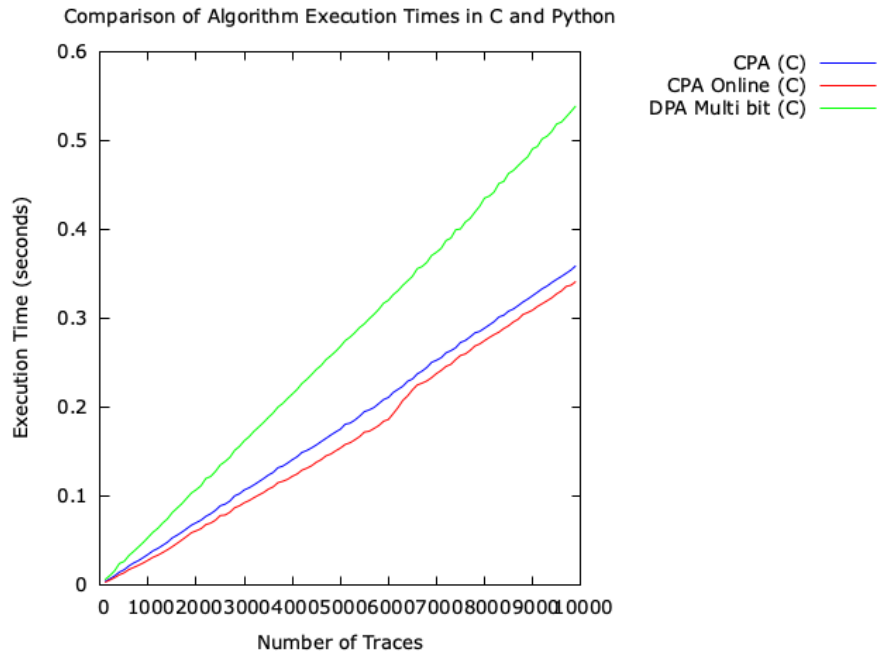


Figure 2: Comparaison des temps d'exécution des algorithmes en C

Pour les implémentations en C, CPA 'online' montre un temps d'exécution inférieur comparé à CPA classique, ce qui est explicable par la fusion des boucles, économisant de la logique de boucle et simplifiant les instructions. Cependant on remarque aussi que le temps d'exécution reste relativement linéaire par rapport au nombre de traces ce qui indique que l'on a pas (ou très peu) de sorties prématurées lors des checkpoints. La DPA multibits quand a elle est considérablement plus lente, ce qui couplé a sa moins bonne efficacité la rends nettement moins intéressante que la CPA.

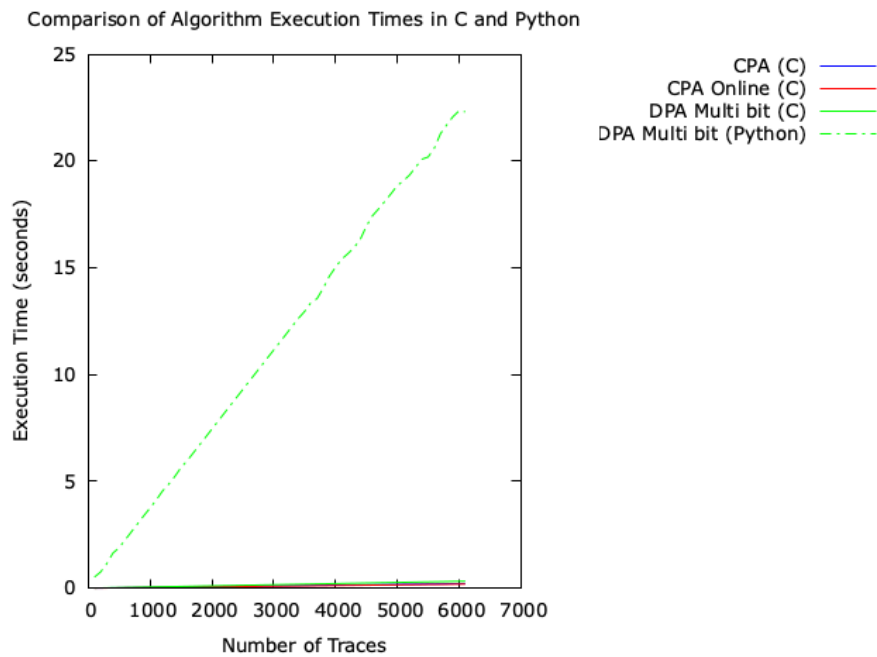


Figure 3: Comparaison des temps d'exécution des algorithmes en C et Python

Table 1: Comparison of Execution Times for Different Algorithms

Algorithm	Language	Execution Time (seconds)
CPA	C	0.358929
CPA Online	C	0.341203
DPA Multibit	C	0.538546
DPA Multibit	Python	36.833080

Le tableau ci-dessus montre que, tandis que le DPA multibits implémenté en C ne prend que 0.538546 secondes pour exécuter 10000 traces, la même algorithme implémenté en Python nécessite 36.833080 secondes. Cette différence significative souligne l'efficacité du C par rapport au Python pour des tâches computationnellement intensives telles que la cryptanalyse. La gestion optimisée de la mémoire et les processus de compilation en C offrent des avantages de performance substantiels, qui sont critiques dans les opérations cryptographiques sensibles au temps.

4 Conclusion

Les résultats de cette étude montrent que le choix de l'algorithme et du langage de programmation a un impact significatif sur les performances des attaques par canaux auxiliaires, tant en termes de taux de réussite que de temps d'exécution.

Du point de vue du taux de réussite, les méthodes CPA et CPA 'online' se distinguent par leur capacité à atteindre un taux de récupération élevé des clés avec un nombre relativement faible de traces. En revanche, bien que le DPA multibits offre une alternative utile dans certains scénarios, il nécessite un plus grand nombre de traces pour atteindre des taux de réussite similaires, ce qui peut le rendre moins pratique dans des environnements limités en données.

En termes de temps d'exécution, les implémentations en C surpassent largement celles en Python. Pour le DPA multibits, l'écart est particulièrement prononcé, avec un temps d'exécution de 0.538546 secondes en C contre 36.833080 secondes en Python pour 10000 traces. Cette différence met en évidence l'efficacité du C pour les tâches computationnellement intensives, tandis que Python, bien qu'offrant une flexibilité et une facilité de développement accrues, se révèle moins adapté pour les environnements nécessitant une exécution rapide.